

Meddly: Multi-terminal and Edge-valued Decision Diagram Library

Junaid Babar and Andrew Miner

Department of Computer Science, Iowa State University, Ames, IA, 50011, USA

Email: {junaid, asminer}@iastate.edu

Abstract

This paper introduces Meddly (Multi-terminal and Edge-valued Decision Diagram Library), an open-source C/C++ library designed from its conception to support Multi-Terminal and Edge-Valued Decision Diagrams. We highlight Meddly's simple and expert interfaces, its loosely-coupled architecture, and the operations available on decision diagrams and conclude with some performance data.

1. Introduction to Meddly

Decision diagrams (DDs) have enjoyed remarkable success in the areas of hardware verification [2], model checking [3], and recently, analysis of stochastic systems [6], [9]. Sophisticated DD algorithms have been developed over the past decade (e.g., [4]). But, the difficult implementation involved with these new approaches—especially those using DD variants such as multi-way DDs (MDDs) [7] and edge-valued DDs (EVMDDs) [8]—has been a significant barrier to their widespread adoption. There are few (if any) DD libraries that support both MDDs and EVMDDs while incorporating newer algorithms. Meddly was developed to address these needs.

1.0.1. Terminology. DDs are directed acyclic graphs used to represent functions on a finite number K of variables, where each variable x_k can assume a finite number n_k of values. Nodes are either *terminal* (no outgoing edges), or are *non-terminal* (labeled with a variable).

All DDs in Meddly eliminate *duplicate* nodes (nodes that represent the same function) and require *ordering* of nodes (there is a total ordering $\succ$ on the variables such that any outgoing edge from a node labeled with x_k must go to either a terminal node or to a node with label x_l with $x_k \succ x_l$). Furthermore, the set of possible values for variable x_k is assumed to be $\mathcal{D}_k = \{0, 1, \dots, n_k - 1\}$. In Meddly, an ordered collection of variables with specified sizes is called a *domain*, which we write as $\mathcal{D} = \mathcal{D}_K \times \dots \times \mathcal{D}_1$.

A named collection of nodes of a particular variety of DD, and associated with a common domain, is called a *forest*. Within a given forest, Meddly automatically eliminates duplicate nodes using a unique table [1], imposes other forest-specific reduction rules, and handles memory management of the nodes (storing them compactly, garbage collection, etc.). Different variable types and ranges, and different rules for managing the graphs lead to various forms of DDs:

- **MDD:** for functions of the form $f : \mathcal{D} \rightarrow \{0, 1\}$.

- **MxD:** matrix diagram [10], $f : \mathcal{D} \times \mathcal{D} \rightarrow \{0, 1\}$. Meddly uses an “interleaved” variable ordering $x_K \succ x'_K \succ \dots \succ x_1 \succ x'_1$.
- **MTMDD:** multi-terminal MDD, $f : \mathcal{D} \rightarrow \mathcal{R}$ with either $\mathcal{R} \subset \mathbb{N}$ or $\mathcal{R} \subset \mathbb{R}$. The function “return values” are stored in the forest within the terminal nodes [5].
- **MTMxD:** multi-terminal MxD, $f : \mathcal{D} \times \mathcal{D} \rightarrow \mathcal{R}$.
- **EV⁺MDD:** edge-valued MDD, $f : \mathcal{D} \rightarrow \mathbb{N} \cup \{\infty\}$. The function “return values” are stored in the forest along the edges and are summed together along paths in the graph [8].
- **EV^{*}MxD:** edge-valued MxD $f : \mathcal{D} \times \mathcal{D} \rightarrow \mathbb{R}$. The function “return values” are stored along the edges and are multiplied together along paths [9].

1.0.2. Native Support for MDDs. Meddly provides “native” support for the above DDs (unlike libraries that store them as BDDs and supply a wrapper to provide the desired behavior). This facilitates efficient use of the “heavier” nodes used by these DDs. To our knowledge, Meddly is the only open-source C/C++ library that natively supports these DDs.

1.0.3. Simple and Expert User Interfaces. One of Meddly's goals is to provide the user with a simple yet powerful interface. For this purpose, it provides two distinct interfaces: a simple interface that provides most of the functionality while minimizing exposure to the underlying components, and an expert interface that provides lower level access.

1.0.4. Extensible DD Variables. DD variables in Meddly can “grow” or “shrink” as directed by the user. Meddly also allows the user to create variables of minimal sizes and let Meddly handle their growth.

1.0.5. Node References. The simple interface works with *node references* to eliminate the need for the user to track and release DD nodes—important steps in controlling memory usage that seemingly confound users of DD libraries. The expert interface while facilitating low-level access to the data structures, operations and compute cache, still permits the use of node references.

1.0.6. Batch Addition. All DD libraries provide operations to build DDs by combining existing DDs. Additionally, Meddly provides efficient batch addition of DD elements (where an element is an assignment of the DD variables).

1.0.7. Built-in Operations. Meddly provides a breadth of built-in operations for all supported DDs. These can be broadly classified as follows:

- **Unary:** Complement, Cardinality, Copy (for “copying” between different types of DDs: $MDD \leftrightarrow MTMDD$, $MxD \leftrightarrow MTMxD$, $MTMDD \leftrightarrow EVMDD$).
- **Binary on booleans:** Union, Intersection, Difference.
- **Binary on integers and reals:** $+$, $-$, $*$, $/$.
- **Relational:** $=$, $\neq$, $<$, $>$, $\leq$, $\geq$, $\min$, $\max$.

Also, given an initial set of states and a next-state function, Meddly can *symbolically* generate the states reachable in one step (Pre-Image and Post-Image) or an infinite number of steps (Reachable States) (using either the “traditional” algorithm [3] or “saturation” [4]).

1.0.8. User-defined Operations. All major components interact via the expert interface. This loose coupling allows user-defined operations to be written as efficiently as the built-in operations. Existing operations can also be *extended*.

1.0.9. Iterators. Iterators, in Meddly, provide an efficient mechanism to cycle through the non-null elements of a DD.

1.0.10. Sparse Nodes. Meddly uses two node representation schemes (Full and Sparse nodes), to reduce memory usage. Meddly’s operations are optimized to work with each of these schemes so there is no penalty to using sparse nodes.

1.0.11. Node Deletion Policy. Meddly uses two node deletion policies: *Pessimistic* and *Optimistic*. Pessimistic deletion marks a node for garbage collection as soon as no other nodes refers to it. Optimistic deletion on the other hand, waits till no node or cache entry refers to it. Pessimistic deletion usually results in lower “peak” memory usage, while Optimistic deletion usually results in lower computation time. The user can also run the garbage collector to delete “stale” nodes and compute cache entries.

1.0.12. Reduction Rules. Meddly provides these reduction rules: Fully, Quasi and Identity [4]. DDs that store relations default to Identity-Reduced and the rest to Fully-Reduced.

2. Conclusions and Future Work

In our opinion, Meddly’s native support for MDDs, EVMDDs and their variants, combined with its performance makes Meddly a good candidate for applications in areas where these DDs make sense. Meddly has already been integrated into GreatSPN and SMART. In both cases, only the simple interface was needed (in our opinion, a testament to its easy of use and the breadth of available operations).

In the short-term we plan on adding operations for efficiently computing the numerical solution of Markov Chains, and the capability for the reordering of domain variables based on existing heuristics.

Meddly is available at <http://meddly.sourceforge.net>.

Acknowledgment

This work is supported in part by the National Science Foundation under grant CNS-0546041.

References

- [1] K. S. Brace, R. L. Rudell, and R. E. Bryant, “Efficient implementation of a BDD package,” in *Conference proceedings on 27th ACM/IEEE design automation conference*. ACM Press, 1990, pp. 40–45.
- [2] R. E. Bryant, “Graph-based algorithms for boolean function manipulation,” *IEEE Trans. Comput.*, vol. C-35, no. 8, pp. 677–691, Aug. 1986.
- [3] J. R. Burch, E. M. Clarke, K. L. McMillan, D. L. Dill, and L. J. Hwang, “Symbolic model checking: 10^{20} states and beyond,” *Information and Computation*, vol. 98, no. 2, pp. 142–170, Jun. 1992.
- [4] G. Ciardo, G. Lüttgen, and A. S. Miner, “Exploiting interleaving semantics in symbolic state-space generation,” *Formal Methods in System Design*, vol. 31, no. 1, pp. 63–100, Aug. 2007.
- [5] M. Fujita, P. McGeer, and J.-Y. Yang, “Multi-terminal binary decision diagrams: An efficient data structure for matrix representation,” *Formal Methods in System Design*, vol. 10, no. 2–3, pp. 149–169, Apr. 1997.
- [6] H. Hermanns, M. Kwiatkowska, G. Norman, D. Parker, and M. Siegle, “On the use of MTBDDs for performability analysis and verification of stochastic systems,” *Journal of Logic and Algebraic Programming: Special Issue on Probabilistic Techniques for the Design and Analysis of Systems*, vol. 56, no. 1–2, pp. 23–67, 2003.
- [7] T. Kam, T. Villa, R. Brayton, and A. Sangiovanni-Vincentelli, “Multi-valued decision diagrams: theory and applications,” *Multiple-Valued Logic*, vol. 4, no. 1–2, pp. 9–62, 1998.
- [8] Y.-T. Lai, M. Pedram, and S. Vrudhula, “Formal verification using edge-valued binary decision diagrams,” *IEEE Trans. Comput.*, vol. 45, no. 2, pp. 247–255, Feb 1996.
- [9] A. S. Miner, “Efficient solution of GSPNs using Canonical Matrix Diagrams,” in *9th Int. Workshop on Petri Nets and Performance Models (PNPM’01)*. IEEE Comp. Soc. Press, Sep. 2001, pp. 101–110.
- [10] —, “Implicit GSPN reachability set generation using decision diagrams,” *Perf. Eval.*, vol. 56, no. 1–4, pp. 145–165, Mar. 2004.